Lab 3: Architecture Selection & Justification

Architecture Selection: We chose **Layered Architecture** for the **Local Sports Tournament & Bracket Manager** System.

Architectural Choice

The system is organised into three horizontal layers: a **Presentation Layer** (Web UI), a **Business Layer** (Auth & Access Control, Tournament Manager, Match & Score Service, Standings & Tiebreaker Engine) and a **Data Layer** (Database). Each layer talks only to the layer directly below it, through explicit provided/required interfaces.

Why not the alternatives?

Microservices would add operational complexity (service discovery, inter-service network latency, distributed data consistency) that a local tournament platform with two user roles does not need. **Plain Client-Server** offers little internal structure, so bracket logic, scoring and standings would blur together on the server and be harder to change independently.

Two Reasons for the Choice

1. Strong separation of concerns matches the domain. Bracket generation (FR-001), score recording and winner advancement, and standings/tiebreaker calculation are distinct responsibilities. In a layered design each lives in its own business component (Tournament Manager, Match & Score Service, Standings Engine) and they collaborate through small interfaces (IBracketAdvance, IStandingsUpdate). A change to tiebreaker rules touches only the Standings component, and a different bracket format can be added without touching the UI.

2. Simple, replaceable components for a small team and a small deployment. Only two actors (Team Captain and Tournament Director) use the system, usually from a browser. A single deployable backend with a shared database keeps match, bracket and standings data strongly consistent, which matters because a submitted score must immediately and correctly update the bracket and the leaderboard. Because components communicate only through interfaces, any one of them (for example the database) can be replaced without changing its consumers.

Security Advantage

Because the Web UI never accesses the Database directly, every request must pass through the Business Layer, where the **Auth & Access Control** component enforces role-based permissions (only a Tournament Director can create or modify brackets; a Team Captain can submit only the scores of their own matches). Only the Data Layer holds database credentials and builds queries, and it uses parameterised queries, which limits SQL-injection exposure. This gives a single, auditable enforcement point instead of scattered checks.

Performance Benefit

The requirement that standings and the leaderboard recalculate within 200 ms of score submission (NFR-001) is supported by the layered structure: the Match & Score Service calls the Standings Engine in-process through IStandingsUpdate, so there is no network hop between them (unlike microservices), and the Standings Engine reaches the database through a single data interface using indexed queries. Having all calculation in one layer also makes it straightforward to add caching of computed standings and to benchmark that one component against the target latency.